Smart Agriculture System

Database Schema Documentation

Database: SmartAgricultureDB

Engine: MySQL

1. Overview

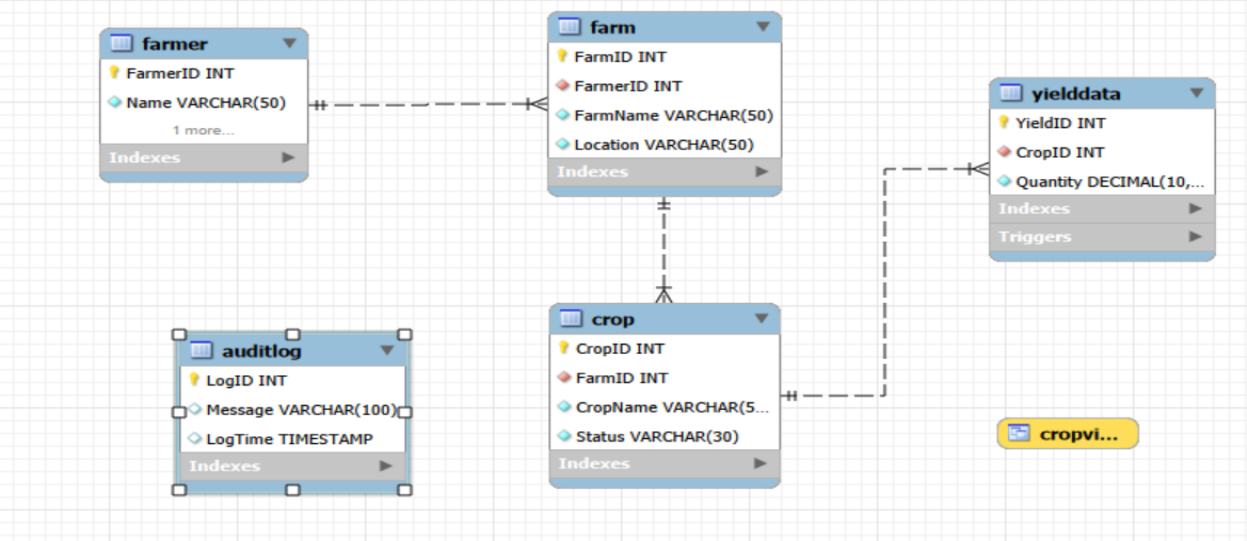
SmartAgricultureDB is a relational database designed to support a smart agriculture monitoring system. It tracks farmers, the farms they own, the crops grown on those farms, recorded yield data for each crop, and a system audit trail of key data-entry events.

The schema consists of five core tables, one view, one stored procedure, and one trigger, described in detail in the sections below.

1.1 Entity Summary

Entity	Purpose	Row Count (sample data)
`Farmer`	Stores farmer identity and contact information	4
`Farm`	Stores farms, each linked to an owning farmer	4
`Crop`	Stores crops grown on a farm and their status	4
`YieldData`	Stores harvest quantity records per crop	4 (5 after trigger test)
`AuditLog`	System-generated log of yield-entry events	Auto-populated by trigger

2. Entity-Relationship Overview



Legend: (1) = one side of the relationship, (M) = many side, —< = one-to-many foreign key relationship.

3. Table Definitions

3.1 Farmer

Stores the core identity details of each registered farmer using the system.

Column	Data Type	Constraints	Description	Notes
FarmerID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for a farmer	Referenced by Farm
Name	VARCHAR(50)	NOT NULL	Full name of the farmer	—
Contact	VARCHAR(20)	NOT NULL	Farmer's phone number	—

3.2 Farm

Stores farms registered in the system, each owned by exactly one farmer.

Column	Data Type	Constraints	Description	Notes
FarmID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for a farm	Referenced by Crop
FarmerID	INT	NOT NULL, FOREIGN KEY → Farmer(FarmerID)	Owning farmer	Many-to-one to Farmer
FarmName	VARCHAR(50)	NOT NULL	Display name of the farm	—
Location	VARCHAR(50)	NOT NULL	City/region where the farm is located	—

3.3 Crop

Stores crops planted on a farm along with their current growth status.

Column	Data Type	Constraints	Description	Notes
CropID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for a crop record	Referenced by YieldData
FarmID	INT	NOT NULL, FOREIGN KEY → Farm(FarmID)	Farm the crop belongs to	Many-to-one to Farm
CropName	VARCHAR(50)	NOT NULL	Name of the crop (e.g. Wheat, Rice)	—
Status	VARCHAR(30)	NOT NULL	Current status (e.g. Growing, Healthy, Harvested)	Free-text field, not an ENUM

3.4 YieldData

Stores harvest quantity records for a given crop.

Column	Data Type	Constraints	Description	Notes
YieldID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for a yield record	—
CropID	INT	NOT NULL, FOREIGN KEY → Crop(CropID)	Crop this yield belongs to	Many-to-one to Crop
Quantity	DECIMAL(10,2)	NOT NULL	Harvested quantity	Fires YieldLog trigger on insert

3.5 AuditLog

A system-maintained log capturing key events, currently populated only by the YieldLog trigger.

Column	Data Type	Constraints	Description	Notes
LogID	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for a log entry	—
Message	VARCHAR(100)	—	Description of the logged event	e.g. 'New Yield Added'
LogTime	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time the event was logged	Auto-populated

4. Relationships (Foreign Keys)

Child Table	Foreign Key Column	Parent Table	Relationship
Farm	FarmerID	Farmer(FarmerID)	One farmer → many farms
Crop	FarmID	Farm(FarmID)	One farm → many crops
YieldData	CropID	Crop(CropID)	One crop → many yield records

5. Views

5.1 CropView

A simplified, read-only view exposing only the crop name and status, useful for dashboards that don't need internal IDs.

```
CREATE OR REPLACE VIEW CropView AS
SELECT CropName, Status
FROM Crop;
```

6. Stored Procedures

6.1 CropReport

Returns a report of every crop with its ID, name, and current status. Intended for reporting/dashboard use so the query logic lives in the database rather than the application layer.

```
CREATE PROCEDURE CropReport()
BEGIN
    SELECT CropID, CropName, Status
    FROM Crop;
END
```

Usage: CALL CropReport();

7. Triggers

7.1 YieldLog

Fires automatically after every insert into YieldData, writing an entry into AuditLog. This provides a lightweight audit trail whenever new yield data is recorded, without requiring the application to log the event explicitly.

Property	Value	Description
Timing	AFTER INSERT	Runs after the row is committed to YieldData
Table	YieldData	Table being monitored
Scope	FOR EACH ROW	Executes once per inserted row
Action	INSERT INTO AuditLog	Adds the fixed message 'New Yield Added'

```
CREATE TRIGGER YieldLog
AFTER INSERT ON YieldData
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (Message)
    VALUES ('New Yield Added');
END
```

8. Sample Data Summary

The seed data included with this schema represents four farmers, each with one farm, one crop, and one yield record, illustrating the intended 1-1-1-1 chain for demonstration purposes (in production, each farmer may own multiple farms, each farm multiple crops, and so on).

Farmer	Farm	Crop	Status	Yield (kg)
Ali Khan	Rawal Mango Farm (Multan)	Wheat	Harvested*	1,200.00
Ahmed Raza	Ayub Agri Farm (Lahore)	Rice	Growing	1,800.00

Farmer	Farm	Crop	Status	Yield (kg)
Sara Malik	Khagga Farm (Faisalabad)	Cotton	Healthy	950.00
Usman Tariq	Fresh Valley Farm (Bahawalpur)	Maize	Harvested	1,500.00

** Wheat's status was updated from 'Healthy' to 'Harvested' via an UPDATE statement in the source script.*

9. Notes for Developers

- All primary keys use AUTO_INCREMENT INT and all foreign keys enforce referential integrity at the database level.
- Crop.Status is a free-text VARCHAR rather than a constrained ENUM — application code should validate allowed values (e.g. Growing, Healthy, Harvested) before insert.
- AuditLog should not be written to directly by the application; it is intended to be populated exclusively by the YieldLog trigger.
- CropView and CropReport() both exist to give reporting tools a simplified interface without needing to join across Farmer/Farm/Crop directly.